Tasca S2.01. Nocions bàsiques SQL

Comentaris generals del P2P que s'han incorporat a la revisió de l'entrega final:

- Ser consistent amb la utilització d'àlies.
- Incloure només les consultes finals que responen al que demana l'exercici i assegurar una coherència entre el codi sql i les captures que es mostres al pdf.
- Ajustar varchars per no ocupar més espai del que correspon.

NIVELL 1

Exercici 1

A partir dels documents adjunts (estructura_dades i dades_introduir), importa les dues taules. Mostra les característiques principals de l'esquema creat i explica les diferents taules i variables que existeixen. Assegura't d'incloure un diagrama que il·lustri la relació entre les diferents taules i variables.

S'obre i s'executa el fitxer "N1-Ex.1__ estructura dades"

Output

Action Output

#	Time	Action	Message
1	01:14:44	CREATE DATABASE IF NOT EXISTS transactions	1 row(s) affected
2	01:14:44	USE transactions	0 row(s) affected
3	01:14:44	CREATE TABLE IF NOT EXISTS company (id VARCHAR(15) PRIMARY KEY, company_name VARCHAR(255), phone VARCHAR(15...	0 row(s) affected
4	01:14:44	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(255) PRIMARY KEY, credit_card_id VARCHAR(15) REFERENCES credit_card...	0 row(s) affected

Es comprova que s'han creat les taules i els camps que contenen

```
14    -- S'obre i s'executa el fitxer "N1-Ex.1__ estructura dades"
15    -- Es comprova que s'han creat les taules i els camps que contenen
16    DESCRIBE company;
17    DESCRIBE transaction;
18
19    -- S'obre i s'executa el fitxer "N1-Ex.1__dades_introduir"
20    -- Es comprova que les dades de les taules "commpany" i "transaction" s'han introduït correctament
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id	varchar(15)	NO	PRI	NULL	
company_name	varchar(255)	YES		NULL	
phone	varchar(15)	YES		NULL	
email	varchar(100)	YES		NULL	
country	varchar(100)	YES		NULL	
website	varchar(255)	YES		NULL	

Result 3

Output

Action Output

#	Time	Action	Message
1	01:14:44	CREATE DATABASE IF NOT EXISTS transactions	1 row(s) affected
2	01:14:44	USE transactions	0 row(s) affected
3	01:14:44	CREATE TABLE IF NOT EXISTS company (id VARCHAR(15) PRIMARY KEY, company_name VARCHAR(255), phone VARCHAR(1...	0 row(s) affected
4	01:14:44	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(255) PRIMARY KEY, credit_card_id VARCHAR(15) REFERENCES credit_car...	0 row(s) affected
5	01:16:45	DESCRIBE company	6 row(s) returned

```
-- S'obre i s'executa el fitxer "N1-Ex.1__ estructura dades"
-- Es comprova que s'han creat les taules i els camps que contenen
DESCRIBE company;
DESCRIBE transaction;

-- S'obre i s'executa el fitxer "N1-Ex.1__dades_introduir"
-- Es comprova que les dades de les taules "commpany" i "transaction" s'han introduït correctament
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id	varchar(255)	NO	PRI	NULL	
credit_card_id	varchar(15)	YES		NULL	
company_id	varchar(20)	YES	MUL	NULL	
user_id	int	YES		NULL	
lat	float	YES		NULL	
longitude	float	YES		NULL	
timestamp	timestamp	YES		NULL	
amount	decimal(10,2)	YES		NULL	
declined	tinyint(1)	YES		NULL	

Result 4

Output

Action Output

#	Time	Action	Message
1	01:14:44	CREATE DATABASE IF NOT EXISTS transactions	1 row(s) affected
2	01:14:44	USE transactions	0 row(s) affected
3	01:14:44	CREATE TABLE IF NOT EXISTS company (id VARCHAR(15) PRIMARY KEY, company_name VARCHAR(255), phone VARCHAR(1...	0 row(s) affected
4	01:14:44	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(255) PRIMARY KEY, credit_card_id VARCHAR(15) REFERENCES credit_car...	0 row(s) affected
5	01:16:45	DESCRIBE company	6 row(s) returned
6	01:18:25	DESCRIBE transaction	9 row(s) returned

S'obre i s'executa el fitxer "N1-Ex.1__dades_introduir"

Es comprova que les dades de les taules "company" i "transaction" s'han introduït correctament

```
SELECT *
FROM company;

SELECT *
FROM transaction;


-- Exercici 2
-- Utilitzant JOIN realitzaràs les següents consultes:
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	company_name	phone	email	country	website
b-2222	Ac Fermentum Incorporated	06 85 56 52 33	donec.porttitor.tellus@yahoo.net	Germany	https://instagram.com/site
b-2226	Magna A Neque Industries	04 14 44 64 62	risus.donec.nibh@icloud.org	Australia	https://whatsapp.com/group/9
b-2230	Fusce Corp.	08 14 97 58 85	risus@protonmail.edu	United States	https://pinterest.com/sub/cars
b-2234	Convallis In Incorporated	06 66 57 29 50	mauris.ut@aol.couk	Germany	https://cnn.com/user/110
b-2238	Ante Iaculis Nec Foundation	08 23 04 99 53	sed.dictum.proin@outlook.ca	New Zealand	https://netflix.com/settings
b-2242	Donec Ltd	01 25 51 37 37	at.iaculis@hotmail.couk	Norway	https://nytimes.com/user/110
b-2246	Sed Nunc Ltd	02 62 64 73 48	nibh@yahoo.org	United Kingdom	https://cnn.com/one
b-2250	Amet Nulla Donec Corporation	07 15 25 14 74	mattis.integer.eu@protonmail.net	Italy	https://netflix.com/sub/cars
b-2254	Nascetur Ridiculus Mus Inc.	06 26 87 61 84	suspendisse.dui@icloud.net	United States	https://ebay.com/sub
b-2258	Vestibulum Lorem PC	02 02 87 33 40	aenean.massa.integer@aol.net	Belgium	https://pinterest.com/sub/cars

company 5

Output

Action Output

#	Time	Action	Message
1	01:30:50	SELECT * FROM company	100 row(s) returned

```
24 • SELECT *
25   FROM transaction;
26
27
28   -- Exercici 2
29   -- Utilitzant JOIN realitzaràs les següents consultes:
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
00043A49-2949-494B-A5DD-A5BAE3BB19DD	CcS-9294	b-2458	4713	46.1999	1.43554	2024-08-28 07:16:46	395.43	0
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	CcS-5019	b-2370	438	41.5972	12.2218	2016-12-21 20:07:18	155.63	0
00045D6B-ED2E-4F2F-8186-CEE074D875D0	CcS-6699	b-2390	2118	29.7573	-95.3796	2020-07-14 15:37:45	326.01	0
000481C3-1C26-4FEF-83A0-4CD0EB004BBD	CcS-6696	b-2230	2115	53.5489	-113.503	2017-09-04 19:44:53	161.60	0
00051AA4-9CBE-4268-B070-C38062A1B3E2	CcS-7606	b-2266	3025	52.2084	5.69081	2017-01-05 18:19:25	148.91	0
0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	CcU-3358	b-2598	215	53.5535	-113.499	2023-09-23 04:51:43	294.59	0
0009A151-9BCF-4E31-9053-A468FF77FAAB	CcS-7509	b-2546	2928	51.9362	5.34265	2023-12-31 00:06:36	383.63	0
0009D494-6245-4DF9-955D-2C084191CFFB	CcS-8483	b-2526	3902	45.492	-73.5706	2017-07-18 07:52:02	197.80	0
000A1DEC-CDB6-4AB2-A619-71DAB8D4A262	CcS-6467	b-2558	1886	55.7425	-3.30009	2018-09-08 05:29:58	339.94	0
000A1F64-1414-4080-9D92-5678A4D958F2	CcS-5966	b-2550	1385	52.0821	5.28424	2022-09-17 04:02:19	369.71	0

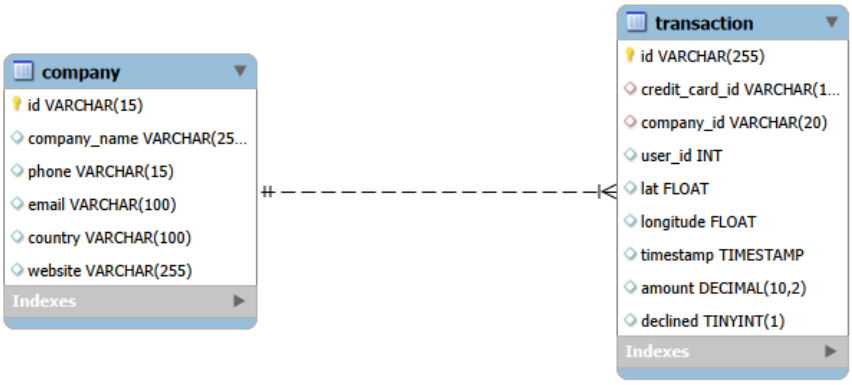
transaction 6

Output

Action Output

#	Time	Action	Message
1	01:30:50	SELECT * FROM company	100 row(s) returned
2	01:31:24	SELECT * FROM transaction	100000 row(s) returned

Es crea un diagrama de la base de dades usant Reverse Engineer



Executant els dos fitxers sql de l'exercici s'han creat les taules "company" i "transaction":

- La taula "company" té un id per a cada empresa, que és la clau primària de cada registre i la resta de camps proporcionen informació sobre aquella empresa (telèfon, email, país, etc.)
- La taula "transaction" té un id per a cada transacció feta, aquesta és la clau primària de la taula i la resta de camps contenen els detalls de cada transacció (targeta de crèdit, empresa, quantitat, si s'ha declinat, etc.). En aquest cas, els detalls no són només "VARCHAR" com en la taula d'empresa, sinó que trobem la data en format TIMESTAMP, les quantitats en número decimal o la opció de declined com a BOOLEAN.
- La taula "transaction" té com a clau forana "company_id", que relaciona cada empresa amb les transaccions que ha fet en una relació 1:N

Exercici 2

Utilitzant JOIN realitzaràs les següents consultes:

Llistat dels països que estan generant vendes.

```sql
-- Llistat dels països que estan generant vendes.
SELECT DISTINCT c.country
FROM company AS c
JOIN transaction AS t
    ON c.id = t.company_id
WHERE t.declined = 0
ORDER BY c.country;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

country
China
France
Germany
Ireland
Italy
Netherlands
New Zealand
Norway
Spain
Sweden

Result 7

Output

Action Output

#	Time	Action	Message
1	01:33:19	SELECT DISTINCT c.country FROM company AS c JOIN transaction AS t ON c.id = t.company_id WHERE t.declined = 0 ORDER BY c.country	15 row(s) returned

S'utilitza el SELECT DISTINCT per a que no surtin països repetits i es fa un JOIN per veure de quins països hi ha realment transaccions, excloent aquelles que no s'han dut a terme (declined = 0). S'ordena per país per a que sigui més fàcil trobar-ne un en concret dins la llista.

Des de quants països es generen les vendes.

```sql
-- Des de quants països es generen les vendes.
SELECT COUNT(DISTINCT c.country) AS num_paisos
FROM company AS c
JOIN transaction AS t
    ON c.id = t.company_id
WHERE t.declined = 0;


-- Identifica la companyia amb la mitjana més gran de vendes.
SELECT c.company_name, AVG(t.amount) as mitjana_vendes
FROM company AS c
JOIN transaction AS t
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

num_paisos
15

Result 10

Output

Action Output

#	Time	Action	Message
1	01:43:19	SELECT COUNT(DISTINCT c.country) AS num_paisos FROM company AS c JOIN transaction AS t ON c.id = t.company_id WHERE t.declined = 0	1 row(s) returned

Com l'anterior, però en aquest cas no volem la llista de països diferents sinó el recompte, per això s'utilitza el COUNT DISTINCT.

Identifica la companyia amb la mitjana més gran de vendes.

```sql
-- Identifica la companyia amb la mitjana més gran de vendes.
SELECT c.company_name, AVG(t.amount) as mitjana_vendes
FROM company AS c
JOIN transaction AS t
    ON c.id = t.company_id
WHERE t.declined = 0
GROUP BY c.id, c.company_name
ORDER BY mitjana_vendes DESC
LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name	mitjana_vendes
Ac Fermentum Incorporated	284.911333

Result 11

Output

Action Output

#	Time	Action	Message
1	01:43:19	SELECT COUNT(DISTINCT c.country) AS num_paisos FROM company AS c JOIN transaction AS t ON c.id = t.company_id WHERE t.declined = 0	1 row(s) returned
2	01:44:06	SELECT c.company_name, AVG(t.amount) as mitjana_vendes FROM company AS c JOIN transaction AS t ON c.id = t.company_id WHERE t.declin...	1 row(s) returned

S'utilitza AVG i GROUP BY empresa per saber calcular la mitjana per empresa i ORDER BY DESC i LIMIT 1 per ordenar-les de més gran a més petita i limitar la resposta a la primera fila (l'empresa que tindrà la mitjana més gran).

Exercici 3

Utilitzant només subconsultes (sense utilitzar JOIN):

Mostra totes les transaccions realitzades per empreses d'Alemanya.

```sql
SELECT *
FROM transaction
WHERE company_id IN (SELECT id
                     FROM company
                     WHERE country = 'Germany');


-- Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.
SELECT c.company_name
FROM company AS c
WHERE c.id IN (SELECT transaction.company_id FROM transaction
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
00138D3B-206D-4C03-94B7-63A2676EB9B4	CcS-4899	b-2222	318	41.3781	12.447	2020-03-25 10:43:43	426.36	0
0013C1B6-3B84-4D6C-8154-E2B3FEBCA8E9	CcS-5070	b-2222	489	41.3814	2.18176	2020-12-17 18:15:37	316.90	0
00201A11-2E62-44C4-941D-198FC8DB77F0	CcU-3512	b-2222	193	55.5704	-3.65129	2021-01-22 23:44:27	453.04	0
00235618-0A5C-4D49-9DCB-B3A9405D8923	CcS-8137	b-2222	3556	59.8421	18.729	2020-09-09 15:43:19	263.14	0
005A5A7B-1F1A-4B6C-9B15-1625A78C9C38	CcS-8998	b-2222	4417	41.1591	-8.63905	2024-05-15 09:10:11	442.01	0

transaction 12

Output

Action Output

#	Time	Action	Message
1	02:03:03	SELECT * FROM transaction WHERE company_id IN (SELECT id FROM company WHERE country = 'Germany')	13291 row(s) returned

En aquest cas no s'han exclòs les transaccions declinades perquè l'enunciat diu "mostra totes les transaccions realitzades" i s'ha interpretat com "independentment de si s'han acceptat o no".

Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.

```
69    -- Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.
70    SELECT c.company_name
71    FROM company AS c
72    WHERE c.id IN (SELECT transaction.company_id FROM transaction
73                   WHERE amount > (SELECT AVG(amount) FROM transaction));
74
75
76    -- Eliminaran del sistema les empreses que no tenen transaccions registrades, entrega el llistat d'aquestes empreses.
77    SELECT c.company_name
78    FROM company AS c
79    WHERE NOT EXISTS (SELECT* FROM transaction as t WHERE c.id = t.company_id);
80
81
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name
Ac Fermentum Incorporated
Magna A Neque Industries
Fusce Corp.
Convallis In Incorporated
Ante Iaculis Nec Foundation

company 13

Output

Action Output

#	Time	Action	Message
1	02:03:03	SELECT * FROM transaction WHERE company_id IN (SELECT id FROM company WHERE country = 'Germany')	13291 row(s) returned
2	02:04:21	SELECT c.company_name FROM company AS c WHERE c.id IN (SELECT transaction.company_id FROM transaction WHERE amount > (SELECT ...	100 row(s) returned

Retorna totes 100 empreses de la taula, ja que totes han fet una transacció per una quantitat superior a la mitjana global.

Eliminaran del sistema les empreses que no tenen transaccions registrades, entrega el llistat d'aquestes empreses.

Segons el resultat de la consulta anterior, 100/100 empreses han realitzat transaccions, així que la consulta hauria de tornar 0 files.

```
77    SELECT c.company_name
78    FROM company AS c
79    WHERE NOT EXISTS (SELECT* FROM transaction as t WHERE c.id = t.company_id);
80
81
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name

company 14

Output

Action Output

#	Time	Action	Message
1	02:03:03	SELECT * FROM transaction WHERE company_id IN (SELECT id FROM company WHERE country = 'Germany')	13291 row(s) returned
2	02:04:21	SELECT c.company_name FROM company AS c WHERE c.id IN (SELECT transaction.company_id FROM transaction WHERE amount > (SELECT ...	100 row(s) returned
3	02:05:47	SELECT c.company_name FROM company AS c WHERE NOT EXISTS (SELECT* FROM transaction as t WHERE c.id = t.company_id)	0 row(s) returned

S'utilitza una subquery amb "NOT EXISTS" per comprovar si alguna empresa no es troba a la taula de transaccions.

Exercici 4

La teva tasca és dissenyar i crear una taula anomenada "credit_card" que emmagatzemi detalls crucials sobre les targetes de crèdit. La nova taula ha de ser capaç d'identificar de manera única cada targeta i establir una relació adequada amb les altres dues taules ("transaction" i "company"). Després de crear la taula serà necessari que ingressis la informació del document denominat "dades_introduir_credit". Recorda mostrar el diagrama i realitzar una breu descripció d'aquest.

Creació de la taula credit_card

```
88      -- Creació de la taula credit_card
89      CREATE TABLE credit_card (
90          id VARCHAR(15) PRIMARY KEY,
91          iban VARCHAR(50),
92          pan VARCHAR(20),
93          pin VARCHAR(5),
94          cvv VARCHAR(5),
95          expiring_date VARCHAR(30)
96          );
97
98      -- S'obre i s'executa el fitxer "N1-Ex.4__ datos_introducir_credit"
99      -- Es comprova que no hi ha transaccions "orfes" de targeta
100     SELECT COUNT(*) AS targetes_no_trobades
101     FROM transactions t
102     LEFT JOIN credit_cards cc
103         ON t.card_id = cc.id
104     WHERE cc.id IS NULL;
105
106     -- Es crea la constraint per relacionar la credit_card_id de la taula transaction amb la PK de la taula credit_card
```

Output

Action Output

#	Time	Action	Message
1	02:25:41	CREATE TABLE credit_card (id VARCHAR(15) PRIMARY KEY, iban VARCHAR(50), pan VARCHAR(20), pin VARCHAR(5), cvv VARCH...	0 row(s) affected

Es deixen tots els camps com a VARCHAR, inclús el de expiring date perquè no caldrà utilitzar-lo per ordenar dates en els exercicis on es fa servir aquesta taula.

S'obre i s'executa el fitxer "N1-Ex.4__ datos_introducir_credit"

Es comprova que les dades s'han introduït correctament

```
98      -- S'obre i s'executa el fitxer "N1-Ex.4__ datos_introducir_credit"
99      -- Es comprova que les dades s'han introduït correctament
100     SELECT *
101     FROM credit_card;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

id	iban	pan	pin	cvv	expiring_date
CcS-4857	XX485759183529250585077 1	2314242385113924	1819	467	09/27/25
CcS-4858	XX858176813700243609402 5	6582720299715533	3964	817	12/28/28
CcS-4859	XX782693049142355360937 0	8861684536289642	4983	277	11/26/26
CcS-4860	XX555959036883530464529 9	2481155515498459	6876	661	07/27/27
CcS-4861	XX203518287719519162730 7	1308930301149557	5710	398	04/25/26

credit_card 15

Output

Action Output

#	Time	Action	Message
1	02:29:51	SELECT * FROM credit_card	5000 row(s) returned

S'han introduït correctament 5000 targetes de crèdit amb els seus detalls

Es comprova que no hi ha transaccions "orfes" de targeta

```
103     -- Es comprova que no hi ha transaccions "orfes" de targeta
104     SELECT COUNT(*) AS targetes_no_trobades
105     FROM transaction t
106     LEFT JOIN credit_card cc
107         ON t.credit_card_id = cc.id
108     WHERE cc.id IS NULL;
109
110     -- Es crea la constraint per relacionar la credit_card_id de la taula transaction amb la PK de la taula credit_card
111     ALTER TABLE transaction
112     ADD CONSTRAINT fk_Credit_card
113     FOREIGN KEY (credit_card_id)
114     REFERENCES credit_card (id);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

targetes_no_trobades
0

Result 16

Output

Action Output

#	Time	Action	Message
1	02:34:00	SELECT COUNT(*) AS targetes_no_trobades FROM transaction t LEFT JOIN credit_card cc ON t.credit_card_id = cc.id WHERE cc.id IS NULL	1 row(s) returned

Totes les transaccions estan relacionades amb una targeta de la taula crèdit_card, així que podem crear la foreign key constaint sense problemes

Creació de la FK Constraint amb la taula transaction

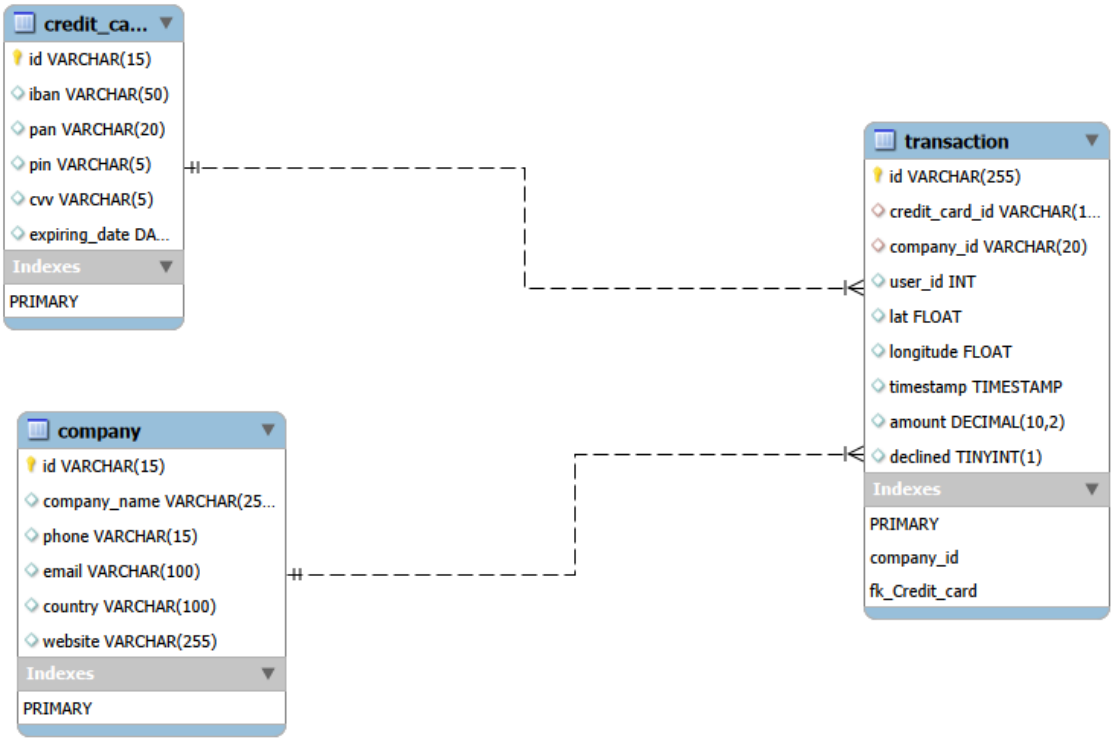
```
110     -- Es crea la constraint per relacionar la credit_card_id de la taula transaction amb la PK de la taula credit_card
111     ALTER TABLE transaction
112     ADD CONSTRAINT fk_Credit_card
113     FOREIGN KEY (credit_card_id)
114     REFERENCES credit_card (id);
115
```

Output

Action Output

#	Time	Action	Message
1	02:35:48	ALTER TABLE transaction ADD CONSTRAINT fk_Credit_card FOREIGN KEY (credit_card_id) REFERENCES credit_card (id)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

Es crea un diagrama de la base de dades usant Reverse Engineer

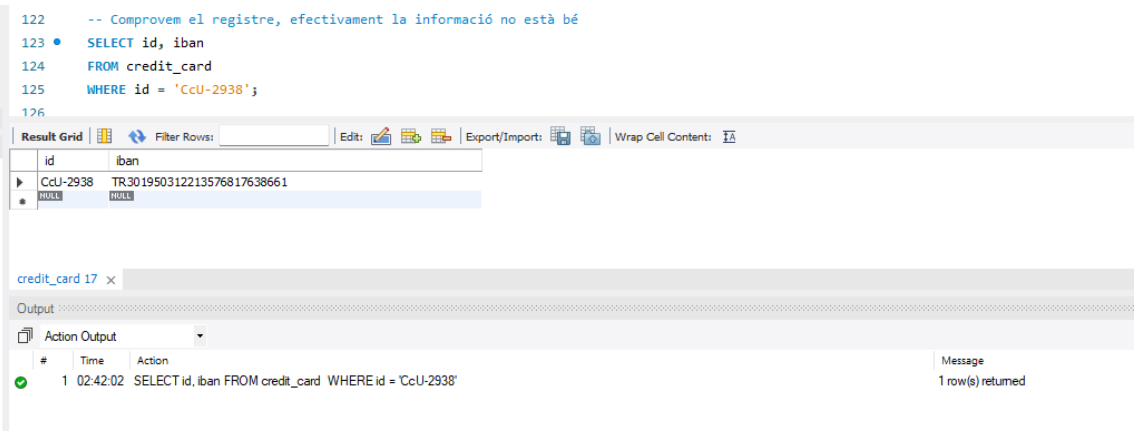


En aquest diagrama podem apreciar com s'ha afegit la taula credit_card a la base de dades amb una relació 1:N amb la taula transaction i la incorporació de la FK per a les targetes de crèdit a la taula transaction (que ja tenia la FK de les empreses d'abans).

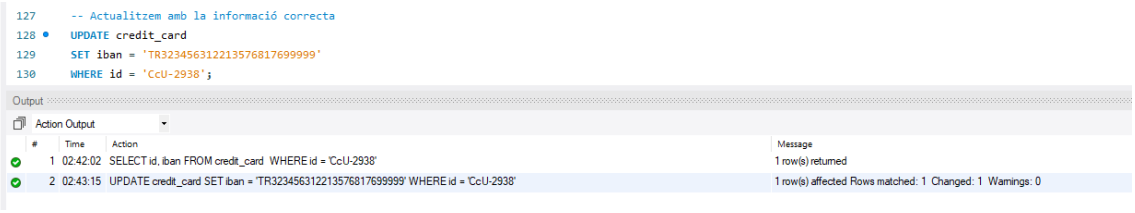
Exercici 5

El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2938. La informació que ha de mostrar-se per a aquest registre és: TR323456312213576817699999. Recorda mostrar que el canvi es va realitzar.

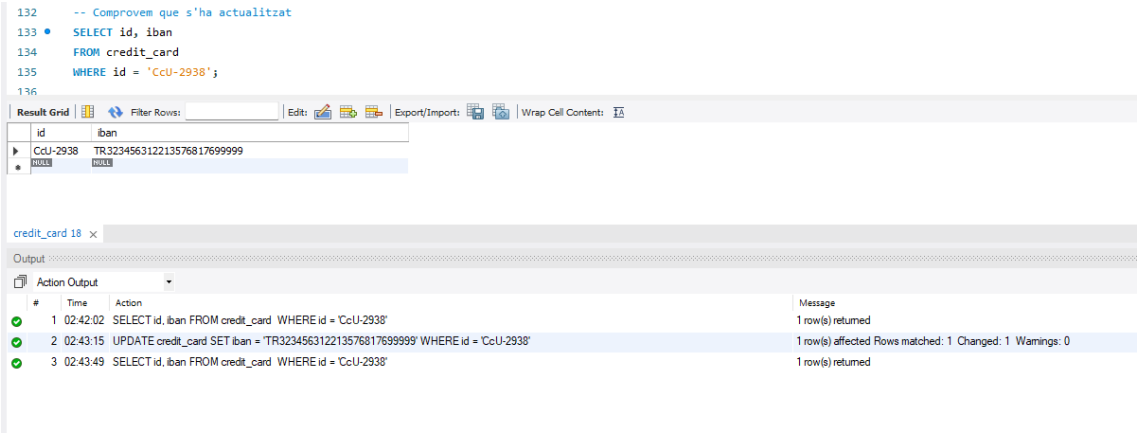
Comprovem el registre, efectivament la informació no està bé



Actualitzem amb la informació correcta



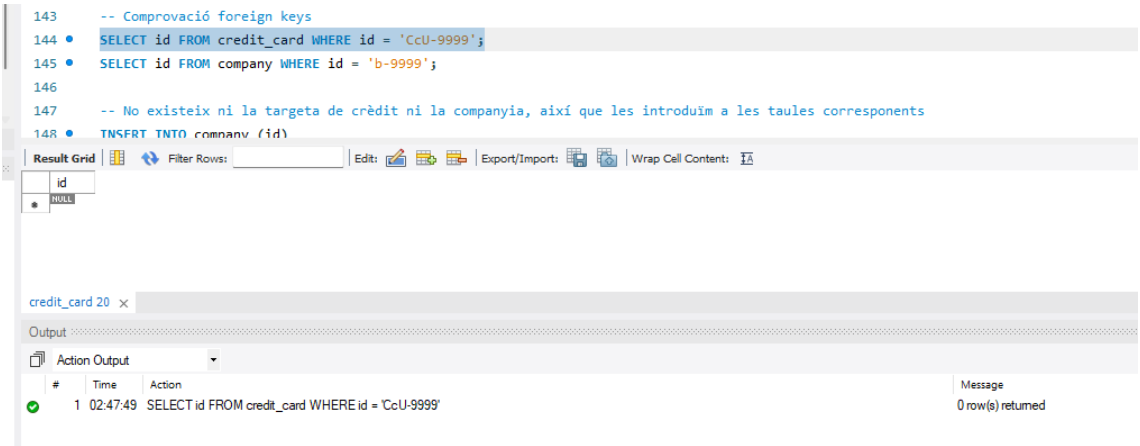
Comprovem que s'ha actualitzat



Exercici 6

En la taula "transaction" ingressa una nova transacció amb la següent informació: Id 108B1D1D-5B23-A76C-55EF-C568E49A99DD; credit_card_id CcU-9999; company_id b-9999; user_id 9999; lat 829.999; longitude -117.999; amount 111.11; declined 0

Comprovació de les claus foranes: l'empresa i la targeta que hem d'introduir existeixen a les respectives taules?



```
143     -- Comprovació foreign keys
144     SELECT id FROM credit_card WHERE id = 'CcU-9999';
145     SELECT id FROM company WHERE id = 'b-9999';
146
147     -- No existeix ni la targeta de crèdit ni la companyia, així que les introduïm a les taules corresponents
148     INSERT INTO company (id)
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id
NULL

company 21

Output

Action Output

#	Time	Action	Message
1	02:47:49	SELECT id FROM credit_card WHERE id = 'CcU-9999'	0 row(s) returned
2	02:48:13	SELECT id FROM company WHERE id = 'b-9999'	0 row(s) returned

Com que ni l'id de l'empresa ni de la targeta de la transacció que s'ha d'introduir existeixen a les respectives taules, es creen els registres referenciats que no existeixen per poder inserir la nova transacció sense vulnerar les restriccions de clau forana.

```
147     -- No existeix ni la targeta de crèdit ni la companyia, així que les introduïm a les taules corresponents
148     INSERT INTO company (id)
149     VALUES ('b-9999');
150
151     INSERT INTO credit_card (id)
152     VALUES ('CcU-9999');
```

Output

Action Output

#	Time	Action	Message
1	02:49:42	INSERT INTO company (id) VALUES ('b-9999')	1 row(s) affected
2	02:49:42	INSERT INTO credit_card (id) VALUES ('CcU-9999')	1 row(s) affected

S'introdueix la transacció que demana l'exercici

```
154     -- I ara ingressem la nova transacció
155     INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('108B1D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', '9999', '829.999', '-117.999', NOW() ,
        '111.11', '0');
156
157     -- Comprovació de que s'ha inserit
```

Output

Action Output

#	Time	Action	Message	Duration / Fe
1	02:49:42	INSERT INTO company (id) VALUES ('b-9999')	1 row(s) affected	0.016 sec
2	02:49:42	INSERT INTO credit_card (id) VALUES ('CcU-9999')	1 row(s) affected	0.000 sec
3	02:51:03	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('108B1D1D-5B23-A76C-55E...	1 row(s) affected	0.015 sec

Es comprova que ha quedat ben introduïda

```
157     -- Comprovació de que s'ha inserit
158     SELECT *
159     FROM transaction
160     WHERE id = '108B1D1D-5B23-A76C-55EF-C568E49A99DD';
161
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
108B1D1D-5B23-A76C-55EF-C568E49A99DD	CcU-9999	b-9999	9999	829.999	-117.999	2026-10-27 02:51:03	111.11	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

transaction 22

Output

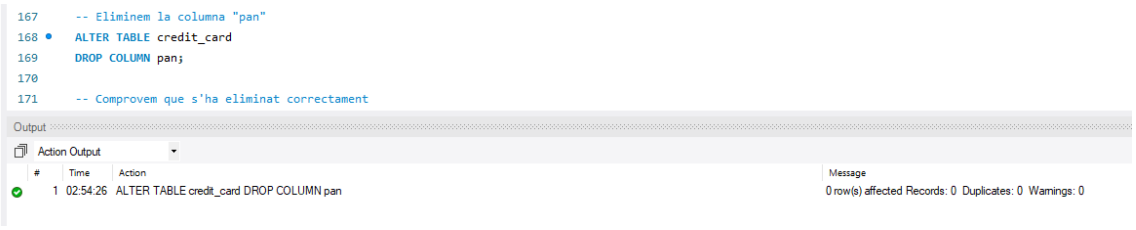
Action Output

#	Time	Action	Message
1	02:49:42	INSERT INTO company (id) VALUES ('b-9999')	1 row(s) affected
2	02:49:42	INSERT INTO credit_card (id) VALUES ('CcU-9999')	1 row(s) affected
3	02:51:03	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('108B1D1D-5B23-A76C-55...	1 row(s) affected
4	02:52:03	SELECT * FROM transaction WHERE id = '108B1D1D-5B23-A76C-55EF-C568E49A99DD'	1 row(s) returned

Exercici 7

Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card. Recorda mostrar el canvi realitzat.

S'elimina la columna "pan"



Es comprova que s'ha eliminat correctament



Exercici 8

Descarrega els arxius CSV que trobaràs a l'apartat de recursos. Estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui almenys 4 taules de les quals puguis realitzar les següents consultes

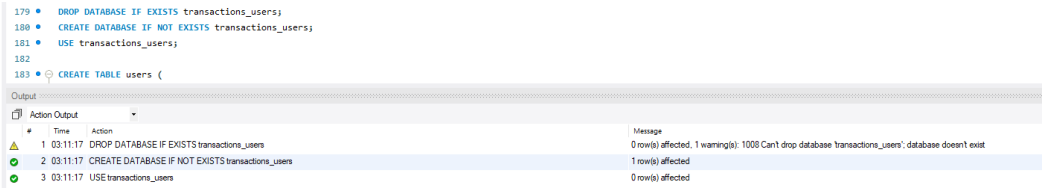
Consideracions abans de la Importació dels fitxers csv:

- Els fitxers CSV originals són necessaris per reproduir la base de dades. Les rutes de LOAD DATA LOCAL INFILE corresponen a l'entorn de desenvolupament i s'han d'adaptar a la ubicació dels CSV. Cal tenir habilitada l'opció local_infile al client i al servidor MySQL.
- Es crea una base de dades nova, `transactions_users`, a partir dels fitxers CSV proporcionats a l'exercici 8. Prèviament, s'ha comparat l'estructura i una mostra dels registres amb les taules existents a la base de dades "transactions". Tot i que els identificadors i les dades comprovades d'empreses i transaccions coincideixen, s'han detectat diferències en els IBAN de les targetes de crèdit, així com columnes addicionals als nous fitxers. Per garantir la coherència de les dades, preservar els

exercicis anteriors i construir l'esquema d'estrella sol·licitat, s'ha optat per crear una base de dades independent, que s'utilitzarà per a la resta d'exercicis de l'Sprint 2.

- S'ha decidit unificar “american_users” i “european_users” en una única taula “users”, ja que totes dues contenen els mateixos camps i representen la mateixa entitat. Tot i que els rangs d'identificadors se solapen, s'ha comprovat que els IDs americans no coincideixen amb els europeus, sinó que es complementen. Per tant, es poden conservar els identificadors originals i utilitzar “id” com a clau primària, simplificant el model i les relacions amb la taula “transactions”. *(Actualització després del P2P):* Per tal de no perdre informació, s’inclou una columna nova “source_region” per poder saber si pertanyien a la taula d’usuaris americans o europeus un cop creada la taula conjunta.
- Els camps “birth_date” i “signup_date” s'han definit com a “VARCHAR(30)” perquè els fitxers originals contenen dates en formats diferents al “DATE” de SQL. Això permet importar-les sense errors de conversió; posteriorment es podrien normalitzar i convertir al tipus “DATE” si fos necessari realitzar operacions amb dates, però revisant l’enunciat dels exercicis posteriors sembla que no caldrà.
- El camp “product_ids” s'ha mantingut temporalment com a text, ja que la relació amb els productes es desenvoluparà en els exercicis posteriors.

Creació de la base de dades transactions_users



Creació de la taula “users”



Comprovació que tenim l'entorn habilitat per carregar dades locals

```
-- Mirem que es puguin carregar dades (això per comprovar el servidor + habilitar-ho al workbench (opcions avançades de la connexió, afegim OPT_LOCAL_INFILE = 1)
SHOW GLOBAL VARIABLES LIKE 'local_infile';
SET GLOBAL local_infile = 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Variable_name	Value
local_infile	ON

Result 24

Output

Action Output

#	Time	Action	Message
1	03:13:14	CREATE TABLE users (id INT PRIMARY KEY, name VARCHAR(100), surname VARCHAR(100), phone VARCHAR(30), email VARCHA...	0 row(s) affected
2	03:15:17	SHOW GLOBAL VARIABLES LIKE 'local_infile'	1 row(s) returned
3	03:15:17	SET GLOBAL local_infile = 1	0 row(s) affected

S'hi introdueixen les dades de l'arxiu "american_users"

```
-- Carreguem taula AMERICAN_USERS
LOAD DATA LOCAL INFILE 'C:/Users/clara/Documents/itacademy/S2/N1-Ex.8__american_users.csv'
INTO TABLE users
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES (id, name, surname, phone, email, birth_date, country, city,
 postal_code, address, signup_date, user_segment, income_band)
SET source_region = 'American';
```

Output

Action Output

#	Time	Action	Message
1	03:16:34	LOAD DATA LOCAL INFILE 'C:/Users/clara/Documents/itacademy/S2/N1-Ex.8__american_users.csv' INTO TABLE users FIELDS TERMINATED B...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0

Comprovació de que s'ha carregat bé

```
-- Comprovem que s'ha carregat bé
SELECT *
FROM users
LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	name	surname	phone	email	birth_date	country	city	postal_code	address	signup_date	user_segment	income_band	source_region
1	Zeus	Gamble	1-282-581-0551	interdum.enim@protonmail.edu	Nov 17, 1985	United States	New York	10001	348-7818 Sagittis St.	2014-04-23	young_professional	high	American
2	Garrett	Mcconnell	(718) 257-2412	integer.vitae.nibh@protonmail.org	Aug 23, 1992	United States	Philadelphia	19101	903 Sit Ave	2014-07-04	young_professional	high	American
3	Ciaran	Harrison	(522) 598-1365	interdum.feugiat@aol.org	Apr 29, 1998	United States	Houston	77001	736-2063 Tellus St.	2014-02-15	frequent_shopper	high	American
4	Howard	Stafford	1-411-740-3269	ornare.egestas@icloud.edu	Feb 18, 1989	United States	Phoenix	85001	Ap #545-2244 Erat. Rd.	2014-08-21	premium	high	American
5	Hayfa	Pierce	1-554-541-2077	et.malesuada.fames@hotmail.org	Sep 26, 1998	United States	Philadelphia	19101	341-2821 Ultrices Av.	2013-04-28	frequent_shopper	high	American

users 25

Output

Action Output

#	Time	Action	Message
1	03:16:34	LOAD DATA LOCAL INFILE 'C:/Users/clara/Documents/itacademy/S2/N1-Ex.8__american_users.csv' INTO TABLE users FIELDS TERMINATED ...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0
2	03:17:24	SELECT * FROM users LIMIT 10	10 row(s) returned

S'hi introdueixen les dades de l'arxiu "european_users"

```
-- Càrrega taula EUROPEAN_USERS
LOAD DATA LOCAL INFILE 'C:/Users/clara/Documents/itacademy/S2/N1-Ex.8__european_users.csv'
INTO TABLE users
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES (id, name, surname, phone, email, birth_date, country, city,
 postal_code, address, signup_date, user_segment, income_band)
SET source_region = 'European';
```

Output

Action Output

#	Time	Action	Message
1	03:18:54	LOAD DATA LOCAL INFILE 'C:/Users/clara/Documents/itacademy/S2/N1-Ex.8__european_users.csv' INTO TABLE users FIELDS TERMINATED B...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0

Comprovem que s'han carregat tots els usuaris d'ambdues taules i que els ids es complementen

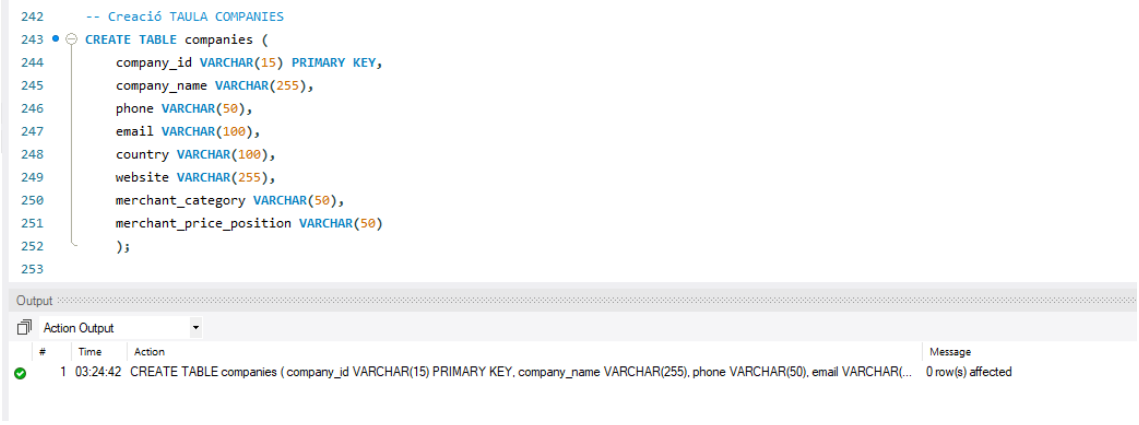


1010 registres de la taula dels americans + 3990 de la taula dels europeus = 5000 usuaris en total, 5000 ids_unics (definitivament no hi havia cap id repetit) i van de l'1 al 5000.

Comprovem que s'ha creat bé la columna "source_region" per mantenir la informació de l'arxiu d'origen



Creació de la taula "companies"



S'hi introdueixen les dades de l'arxiu "companies"

```
254    -- Càrrega taula COMPANIES
255    LOAD DATA LOCAL INFILE 'C:/Users/clara/Documents/itacademy/S2/N1-Ex.8__companies.csv'
256    INTO TABLE companies
257    FIELDS TERMINATED BY ','
258    OPTIONALLY ENCLOSED BY '"'
259    LINES TERMINATED BY '\n'
260    IGNORE 1 LINES;
261
262    -- Creació TAULA CREDIT_CARDS
```

Output

Action Output

#	Time	Action	Message
1	03:24:42	CREATE TABLE companies (company_id VARCHAR(15) PRIMARY KEY, company_name VARCHAR(255), phone VARCHAR(50), email VARCHAR(...	0 row(s) affected
2	03:25:41	LOAD DATA LOCAL INFILE 'C:/Users/clara/Documents/itacademy/S2/N1-Ex.8__companies.csv' INTO TABLE companies FIELDS TERMINATED B...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0

Comprovació de que s'han carregat bé

```
262    -- Comprovem que s'ha carregat bé
263    SELECT *
264    FROM companies
265    LIMIT 10;
266
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_id	company_name	phone	email	country	website	merchant_category	merchant_price_position
b-2222	Ac Fermentum Incorporated	+64 601 244 6320	donec.porttitor.tellus@yahoo.net	Germany	https://www.ac-fermentum-incorporated.com	Beauty	mid-market
b-2226	Magna A Neque Industries	+33 823 560 3779	risus.donec.nibh@icloud.org	Australia	https://www.magna-a-neque-industries.com	Travel	budget
b-2230	Fusce Corp.	+29 657 646 4324	risus@protonmail.edu	United States	https://www.fusce-corp.com	Home	mid-market
b-2234	Convallis In Incorporated	+6 693 982 1255	mauris.ut@aol.couk	Germany	https://www.convallis-in-incorporated.com	Sports	mid-market
b-2238	Ante Iaculis Nec Foundation	+69 914 549 1009	sed.dictum.proin@outlook.ca	New Zealand	https://www.ante-iaculis-nec-foundation.com	Toys	mid-market

companies 28

Output

Action Output

#	Time	Action	Message
1	03:24:42	CREATE TABLE companies (company_id VARCHAR(15) PRIMARY KEY, company_name VARCHAR(255), phone VARCHAR(50), email VARCHAR...	0 row(s) affected
2	03:25:41	LOAD DATA LOCAL INFILE 'C:/Users/clara/Documents/itacademy/S2/N1-Ex.8__companies.csv' INTO TABLE companies FIELDS TERMINATED ...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0
3	03:26:41	SELECT * FROM companies LIMIT 10	10 row(s) returned

Creació de la taula "credit_cards"

```
267    -- Creació TAULA CREDIT_CARDS
268    CREATE TABLE credit_cards (
269        id VARCHAR(15) PRIMARY KEY,
270        user_id INT,
271        iban VARCHAR(50),
272        pan VARCHAR(20),
273        pin VARCHAR(5),
274        cvv VARCHAR(5),
275        track1 VARCHAR(255),
276        track2 VARCHAR(255),
277        expiring_date VARCHAR(30),
278        card_type VARCHAR(30),
279        card_renewal_flag BOOLEAN
280        );
281
282    -- Càrrega taula CREDIT_CARDS
```

Output

Action Output

#	Time	Action	Message
1	03:33:50	CREATE TABLE credit_cards (id VARCHAR(15) PRIMARY KEY, user_id INT, iban VARCHAR(50), pan VARCHAR(20), pin VARCHAR(5), ...	0 row(s) affected

S'hi introdueixen les dades de l'arxiu "credit_cards"

```
282    -- Càrrega taula CREDIT_CARDS
283    LOAD DATA LOCAL INFILE 'C:/Users/clara/Documents/itacademy/S2/N1-Ex.8__credit_cards.csv'
284    INTO TABLE credit_cards
285    FIELDS TERMINATED BY ','
286    OPTIONALLY ENCLOSED BY '"'
287    LINES TERMINATED BY '\n'
288    IGNORE 1 LINES;
```

Output

Action Output

#	Time	Action	Message
1	03:33:50	CREATE TABLE credit_cards (id VARCHAR(15) PRIMARY KEY, user_id INT, iban VARCHAR(50), pan VARCHAR(20), pin VARCHAR(5), ...	0 row(s) affected
2	03:34:26	LOAD DATA LOCAL INFILE 'C:/Users/clara/Documents/itacademy/S2/N1-Ex.8__credit_cards.csv' INTO TABLE credit_cards FIELDS TERMINATE...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0

Comprovació de que s'han carregat bé

```
290     -- Comprovació
291 •   SELECT *
292     FROM credit_cards
293     LIMIT 10;
294
295     -- Creació TAULA TRANSACTIONS
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	user_id	iban	pan	pin	cvv	track1	track2	expiring_date	card_type	card_renewal_flag
CcS-4857	276	ES4156794166769687 11699208	2314242385113924	1819	467	%B2314242385113924^LWCBUDLWCBUD^22...	%B2314242385113924=2410101518363164?	09/25/28	debit	1
CcS-4858	277	NL2520530376098512622422 44	6582720299715533	3964	817	%B6582720299715533^TIQMVITIQMVI^2404...	%B6582720299715533=2411011045462722	12/28/28	debit	0
CcS-4859	278	SE7200843185843647714972 62	8861684536289642	4983	277	%B8861684536289642^COFBGDCOFBGD^280...	%B8861684536289642=2502101761665371?	11/26/28	credit	0
CcS-4860	279	ES1655740780453141365283427	2481155515498459	6876	661	%B2481155515498459^TIUJTUTIUJTU^31040...	%B2481155515498459=2602101514414395?	07/27/28	prepaid	0
CcS-4861	280	SE6567622104310251966208 41	1308930301149557	5710	398	%B1308930301149557^HPOBNZHPOBNZ^330...	%B1308930301149557=2805101751305028?	04/26/28	credit	0

credit_cards 29 ×

Output

Action Output

#	Time	Action	Message
1	03:33:50	CREATE TABLE credit_cards (id VARCHAR(15) PRIMARY KEY, user_id INT, iban VARCHAR(50), pan VARCHAR(20), pin VARCHAR(5)...	0 row(s) affected
2	03:34:26	LOAD DATA LOCAL INFILE 'C:/Users/clara/Documents/itacademy/S2/N1-Ex.8__credit_cards.csv' INTO TABLE credit_cards FIELDS TERMINAT...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0
3	03:34:59	SELECT * FROM credit_cards LIMIT 10	10 row(s) returned

Creació de la taula "transactions"

```
296 •   CREATE TABLE transactions (
297         id VARCHAR(255) PRIMARY KEY,
298         card_id VARCHAR(15),
299         business_id VARCHAR(15),
300         timestamp TIMESTAMP,
301         amount DECIMAL(10, 2),
302         declined BOOLEAN,
303         product_ids TEXT(150),
304         user_id INT,
305         lat FLOAT,
306         longitude FLOAT,
307         discount_amount DECIMAL(10, 2),
308         tax_amount DECIMAL(10, 2),
309         shipping_amount DECIMAL(10, 2),
310         channel VARCHAR(50),
311         campaign_id VARCHAR(50),
312         device_type VARCHAR(50),
313         is_international BOOLEAN,
314         decline_reason VARCHAR(250),
315         distance_km DECIMAL(10, 2)
316         );
317
```

Output

Action Output

#	Time	Action	Message
1	03:35:39	CREATE TABLE transactions (id VARCHAR(255) PRIMARY KEY, card_id VARCHAR(15), business_id VARCHAR(15), timestamp TIMESTAMP, ...	0 row(s) affected

S'hi introdueixen les dades de l'arxiu "transactions"

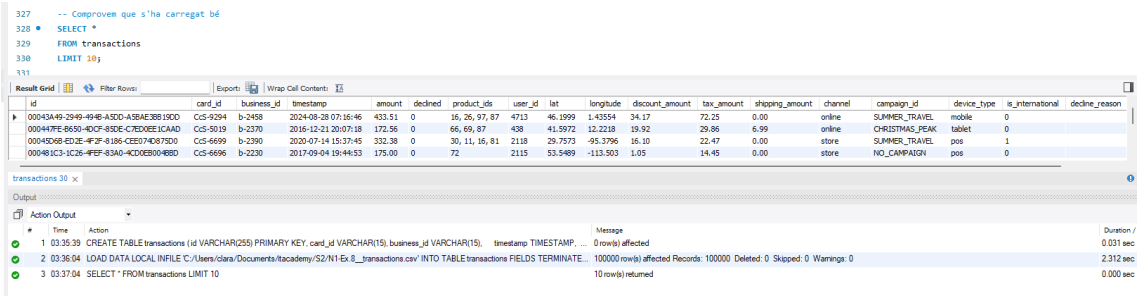
```
320     -- Càrrega taula transactions
321 •   LOAD DATA LOCAL INFILE 'C:/Users/clara/Documents/itacademy/S2/N1-Ex.8__transactions.csv'
322     INTO TABLE transactions
323     FIELDS TERMINATED BY ';'
324     OPTIONALLY ENCLOSED BY '"'
325     LINES TERMINATED BY '\n'
326     IGNORE 1 LINES;
327
328     -- Comprovem que no hi ha registres "orfes" abans de crear FK
329 •   SELECT COUNT(*) AS targetes_no_trobades
```

Output

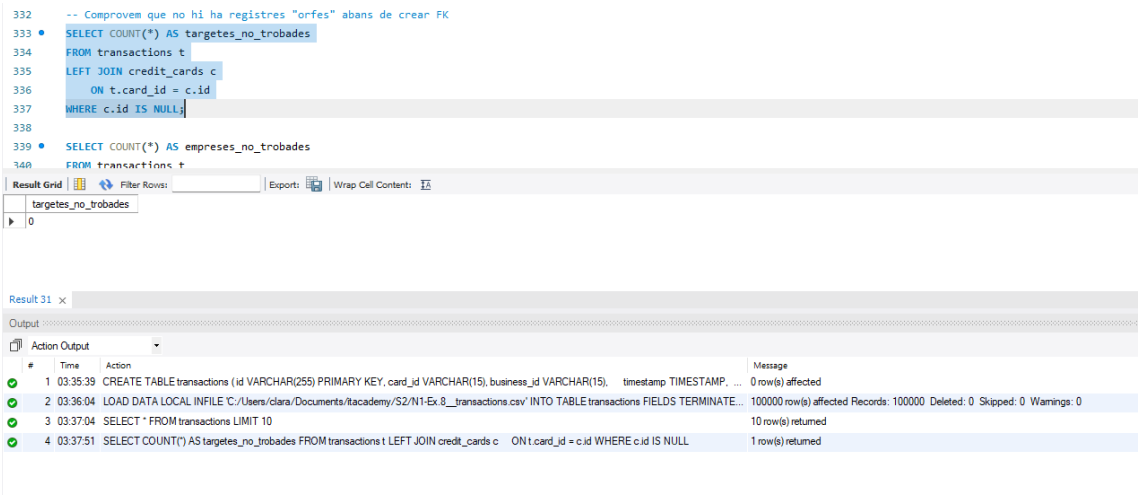
Action Output

#	Time	Action	Message
1	03:35:39	CREATE TABLE transactions (id VARCHAR(255) PRIMARY KEY, card_id VARCHAR(15), business_id VARCHAR(15), timestamp TIMESTAMP, ...	0 row(s) affected
2	03:36:04	LOAD DATA LOCAL INFILE 'C:/Users/clara/Documents/itacademy/S2/N1-Ex.8__transactions.csv' INTO TABLE transactions FIELDS TERMINATE...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0

Comprovació de que s'han carregat bé



Comprovem que no hi ha registres "orfes" a transactions abans de crear FK amb les altres taules



No hi ha cap targeta a "transactions" que no existeixi a la taula "credit_cards"



No hi ha cap empresa a "transactions" que no existeixi a la taula "companies"

```
345 •  SELECT COUNT(*) AS usuaris_no_trobats
346    FROM transactions t
347    LEFT JOIN users u
348        ON t.user_id = u.id
349    WHERE u.id IS NULL;
350
351    -- Creem FK
352 •  ALTER TABLE transactions
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

usuaris_no_trobats
0

Result 33

Output

Action Output

#	Time	Action	Message
1	03:38:21	SELECT COUNT(*) AS empreses_no_trobades FROM transactions t LEFT JOIN companies c ON t.business_id = c.company_id WHERE c.compa...	1 row(s) returned
2	03:40:28	SELECT COUNT(*) AS usuaris_no_trobats FROM transactions t LEFT JOIN users u ON t.user_id = u.id WHERE u.id IS NULL	1 row(s) returned

No hi ha cap usuari a “transactions” que no existeixi a la taula “users”

Creem les claus foranes i constraints

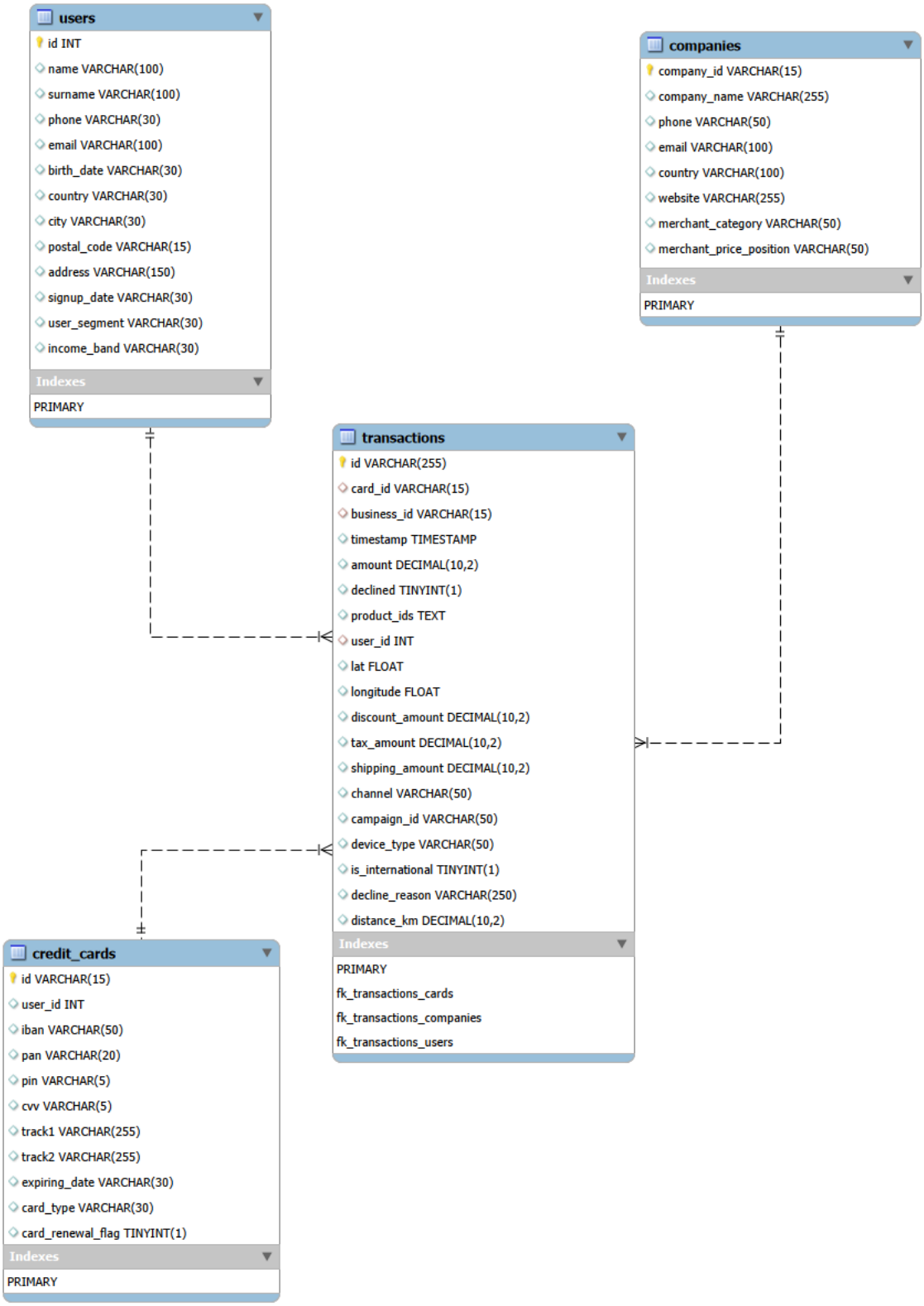
```
351    -- Creem FK
352 •  ALTER TABLE transactions
353    ADD CONSTRAINT fk_transactions_cards
354    FOREIGN KEY (card_id)
355    REFERENCES credit_cards(id);
356
357 •  ALTER TABLE transactions
358    ADD CONSTRAINT fk_transactions_companies
359    FOREIGN KEY (business_id)
360    REFERENCES companies(company_id);
361
362 •  ALTER TABLE transactions
363    ADD CONSTRAINT fk_transactions_users
364    FOREIGN KEY (user_id)
365    REFERENCES users(id);
366
367    -- Exercici 9
368    -- Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.
```

Output

Action Output

#	Time	Action	Message
1	03:38:21	SELECT COUNT(*) AS empreses_no_trobades FROM transactions t LEFT JOIN companies c ON t.business_id = c.company_id WHERE c.compa...	1 row(s) returned
2	03:40:28	SELECT COUNT(*) AS usuaris_no_trobats FROM transactions t LEFT JOIN users u ON t.user_id = u.id WHERE u.id IS NULL	1 row(s) returned
3	03:41:18	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_cards FOREIGN KEY (card_id) REFERENCES credit_cards(id)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
4	03:41:20	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_companies FOREIGN KEY (business_id) REFERENCES companies(company_id)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
5	03:41:21	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_users FOREIGN KEY (user_id) REFERENCES users(id)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

Es crea un diagrama de la base de dades usant Reverse Engineer



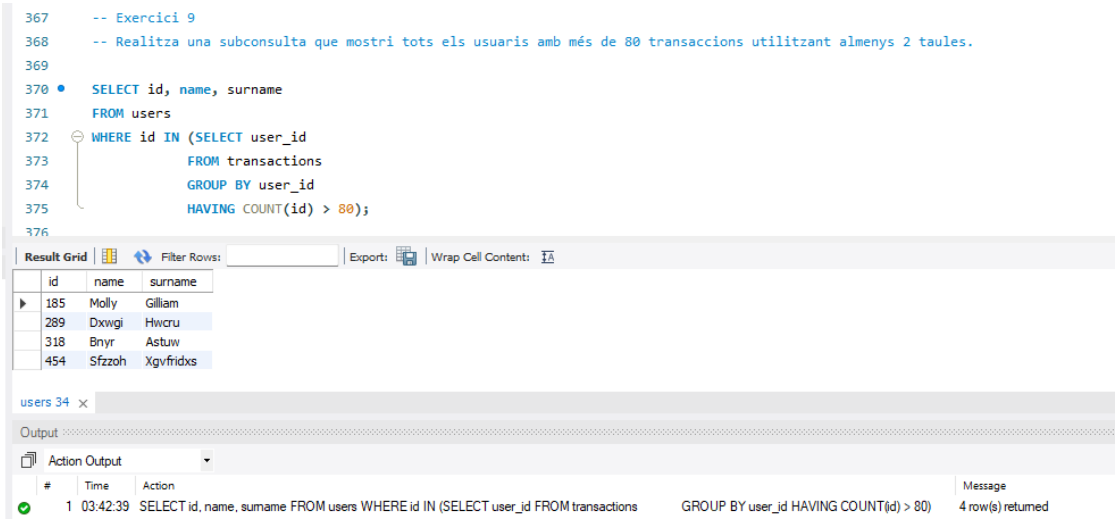
S'ha creat un esquema d'estrella amb “transactions” com a taula de fets i “users”, “companyies” i “credit_cards” com a dimensions. Tot i que “credit_cards” inclou un camp “user_id”, s'ha decidit no establir una relació directa entre aquestes dues dimensions per

mantenir un model analític senzill i evitar relacions addicionals que no són necessàries per a les consultes plantejades. Es conserva el camp original per a possibles anàlisis posteriors.

La taula central transactions conté 100.000 registres i es relaciona mitjançant claus foranes amb les tres taules de dimensions: users (5.000 registres), companies (100 registres) i credit_cards (5.000 registres). S'ha comprovat la integritat referencial de les tres relacions abans de crear les claus foranes.

Exercici 9

Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.



Com que demana transaccions fetes per usuaris, sense especificar, incloem totes les transaccions realitzades encara que fossin finalment declinades. Segons aquesta interpretació, hi ha 4 usuaris que han fet més de 80 transactions

Exercici 10

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

```
377  -- Exercici 10
378  -- Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.
379  SELECT cc.iban, ROUND(AVG(t.amount), 2) as mitjana_amount
380  FROM transactions t
381  JOIN credit_cards cc
382      ON t.card_id = cc.id
383  JOIN companies co
384      ON t.business_id = co.company_id
385  WHERE co.company_name = 'Donec Ltd' AND t.declined = 0
386  GROUP BY cc.iban
387  ORDER BY mitjana_amount DESC;
388
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

iban	mitjana_amount
US212768278745981809500233	708.14
CA817443448587215122047864	690.81
US157275989571099139868887	647.08
US594988240619959562749388	638.82
US186466262266569948042760	629.76

Result 36

Output

Action Output

#	Time	Action	Message
1	03:47:51	SELECT cc.iban, ROUND(AVG(t.amount), 2) as mitjana_amount FROM transactions t JOIN credit_cards cc ON t.card_id = cc.id JOIN companies co ...	367 row(s) returned

Usem JOIN per poder veure la mitjana de quantitat (taula “transactions”) per cada IBAN (taula “credit_cards”) i filtrar per l’empresa que ens demanen (taula “companies”).

NIVELL 2

Exercici 1

Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes. Mostra la data de cada transacció juntament amb el total de les vendes.

```
402 •  SELECT DATE(timestamp) AS data, SUM(amount) AS total_vendes
403    FROM transactions
404    WHERE declined = 0
405    GROUP BY DATE(timestamp)
406    ORDER BY total_vendes DESC
407    LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

data	total_vendes
2023-02-20	15728.40
2019-11-18	15447.89
2022-12-13	14944.00
2019-03-18	14896.47
2017-12-20	14367.96

Result 1

Output

Action Output

#	Time	Action	Message
1	10:02:26	SELECT DATE(timestamp) AS data, SUM(amount) AS total_vendes FROM transactions WHERE declined = ...	5 row(s) returned

Usem SUM(amount) i GROUP BY DATE(timestamp) per obtenir la suma d'ingressos per dia, excloent les transaccions rebutjades, i l'ORDER BY DESC i LIMIT 5 per ordenar-los començant per la quantitat més gran i seleccionar els 5 primers tal com indica l'exercici.

Exercici 2

Presenta el nom, telèfon, país, data i amount, d'aquelles empreses que van realitzar transaccions amb un valor comprès entre 350 i 400 euros i en alguna d'aquestes dates: 29 d'abril del 2015, 20 de juliol del 2018 i 13 de març del 2024. Ordena els resultats de major a menor quantitat.

```
414 •  SELECT co.company_name, co.phone, co.country, DATE(t.timestamp) AS data, t.amount
415    FROM companies co
416    JOIN transactions t
417        ON t.business_id = co.company_id
418    WHERE t.amount BETWEEN 350 AND 400 AND DATE(t.timestamp) IN ('2015-04-29', '2018-07-20', '2024-03-13')
419    ORDER BY t.amount DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name	phone	country	data	amount
Fringilla LLC	+1 150 784 1716	New Zealand	2015-04-29	390.33
Auctor Mauris Vel LLP	+32 133 447 8239	United States	2024-03-13	383.58
Mauris Institute	+75 473 619 9472	Sweden	2015-04-29	369.20
Pede Cum Ltd	+26 625 798 6126	Norway	2018-07-20	358.64
Netus Et Malesuada Ltd	+52 818 402 1638	Netherlands	2024-03-13	352.21

Result 2

Output

Action Output

#	Time	Action	Message
1	10:08:18	SELECT co.company_name, co.phone, co.country, DATE(t.timestamp) AS data, t.amount FROM companies...	6 row(s) returned

Usem un JOIN entre la taula de “companyies” i la de transactions per poder visualitzar els detalls de les empreses que demana l’exercici, filtrant per les quantitats i dates indicades, que es troben a la taula “transactions”. Com que l’enunciat diu “transaccions realitzades”, s’inclouen totes, incloses les que finalment fossin declinades.

Exercici 3

Necessitem optimitzar l'assignació dels recursos i dependrà de la capacitat operativa que es requereixi, per la qual cosa et demanen la informació sobre la quantitat de transaccions que realitzen les empreses, però el departament de recursos humans és exigent i vol un llistat de les empreses on especifiquis si tenen igual o més de 400 transaccions o menys.

```sql
SELECT co.company_id, co.company_name, COUNT(t.id) AS num_transaccions,
CASE
    WHEN COUNT(t.id) >= 400 THEN '400 o més'
    ELSE 'menys de 400'
END AS categoria_transaccions
FROM companies co
LEFT JOIN transactions t
    ON co.company_id = t.business_id
GROUP BY co.company_id, co.company_name
ORDER BY num_transaccions DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_id	company_name	num_transaccions	categoria_transaccions
b-2222	Ac Fermentum Incorporated	2401	400 o més
b-2302	Nunc Interdum Incorporated	1599	400 o més
b-2330	Donec Fringilla PC	1593	400 o més
b-2366	Mauris Institute	1586	400 o més
b-2614	Rutrum Non Inc.	1585	400 o més

Result 3

Output

Action Output

#	Time	Action	Message
1	10:12:57	SELECT co.company_id, co.company_name, COUNT(t.id) AS num_transaccions, CASE WHEN COUNT(...	100 row(s) returned

Utilitzem el CASE per tal de poder crear una columna que classifiqui les empreses segons la condició que demana l’exercici. Fem ORDER BY per poder comprovar que, efectivament, la classificació s’ha fet bé i quan baixem de 400 surt “menys de 400”.

Exercici 4

Elimina de la taula transaction el registre amb ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de dades.

Comprovació de que aquesta transacció existeix

```
441     -- Comprovem que aquesta transacció existeix
442     SELECT *
443     FROM transactions
444     WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
445
446     -- L'eliminem de la taula
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	CcS-5019	b-2370	2016-12-21 20:07:18	172.56	0	66, 69, 87	438	41.5972
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

transactions 5 ×

Output

Action Output

#	Time	Action	Message
1	11:33:51	SELECT * FROM transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) returned

Eliminació de la taula

```
446     -- L'eliminem de la taula
447     DELETE FROM transactions
448     WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
449
450     -- Comprovem que ja no hi és
```

Output

Action Output

#	Time	Action	Message
1	11:33:51	SELECT * FROM transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) returned
2	11:37:28	DELETE FROM transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) affected

Comprovació de que s'ha eliminat

```
450     -- Comprovem que ja no hi és
451     SELECT *
452     FROM transactions
453     WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat	longitude	discount_amount	tax_amount	shi
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

transactions 6 ×

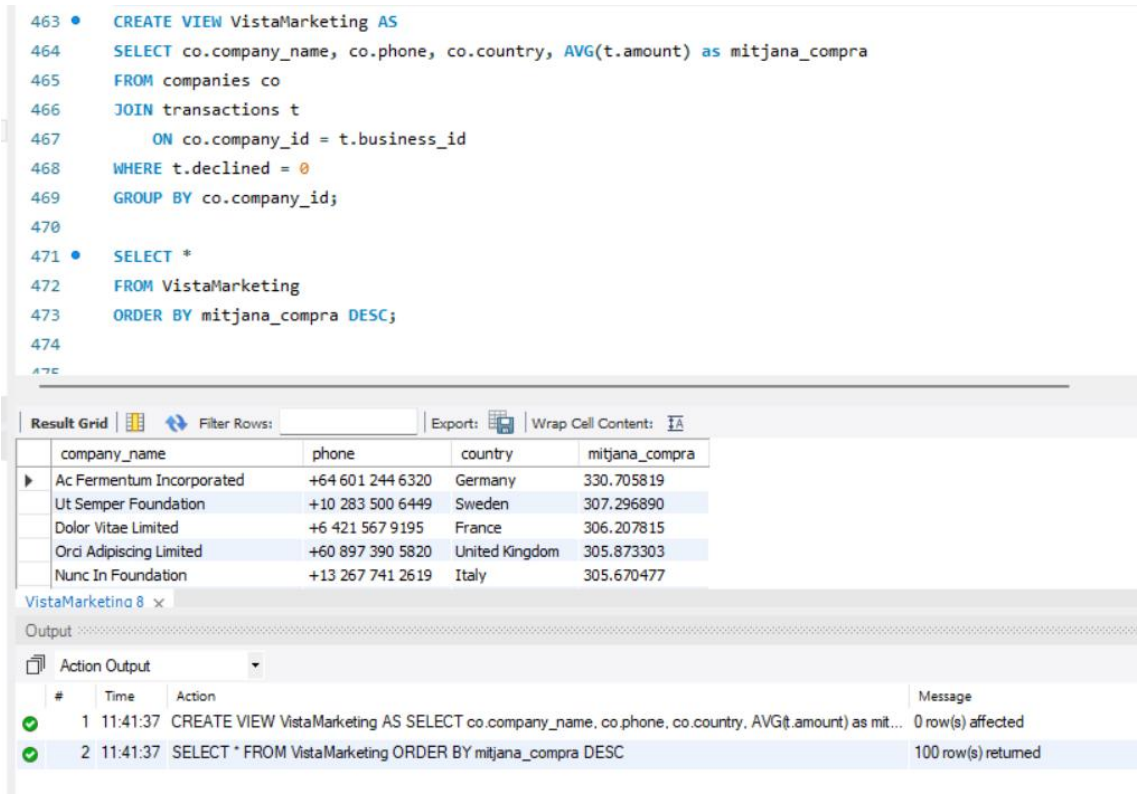
Output

Action Output

#	Time	Action	Message
1	11:33:51	SELECT * FROM transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) returned
2	11:37:28	DELETE FROM transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) affected
3	11:38:13	SELECT * FROM transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	0 row(s) returned

Exercici 5

La secció de màrqueting desitja tenir accés a informació específica per a realitzar anàlisi i estratègies efectives. S'ha sol·licitat crear una vista que proporcioni detalls clau sobre les companyies i les seves transaccions. Serà necessària que creïs una vista anomenada VistaMarketing que contingui la següent informació: Nom de la companyia. Telèfon de contacte. País de residència. Mitjana de compra realitzat per cada companyia. Presenta la vista creada, ordenant les dades de major a menor mitjana de compra.



```
463 •  CREATE VIEW VistaMarketing AS
464    SELECT co.company_name, co.phone, co.country, AVG(t.amount) as mitjana_compra
465    FROM companies co
466    JOIN transactions t
467        ON co.company_id = t.business_id
468    WHERE t.declined = 0
469    GROUP BY co.company_id;
470
471 •  SELECT *
472    FROM VistaMarketing
473    ORDER BY mitjana_compra DESC;
474
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name	phone	country	mitjana_compra
Ac Fermentum Incorporated	+64 601 244 6320	Germany	330.705819
Ut Semper Foundation	+10 283 500 6449	Sweden	307.296890
Dolor Vitae Limited	+6 421 567 9195	France	306.207815
Orci Adipiscing Limited	+60 897 390 5820	United Kingdom	305.873303
Nunc In Foundation	+13 267 741 2619	Italy	305.670477

VistaMarketing 8

Output

Action Output

#	Time	Action	Message
1	11:41:37	CREATE VIEW VistaMarketing AS SELECT co.company_name, co.phone, co.country, AVG(t.amount) as mit...	0 row(s) affected
2	11:41:37	SELECT * FROM VistaMarketing ORDER BY mitjana_compra DESC	100 row(s) returned

Usem CREATE VIEW AS per crear la vista que ens demanen i incloem la consulta per mostrar la taula amb els detalls de l'empresa requerits i la mitjana de compra (s'exclouen les transaccions declinades).

NIVELL 3

Exercici 1

Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon: quantes targetes estan actives?

Abans de crear la taula comprovem que no hi ha targetes sense transaccions.

```sql
-- Comprovem que no hi ha targetes sense transaccions
SELECT COUNT(*) AS targetes_no_usades
FROM credit_cards cc
LEFT JOIN transactions t
    ON cc.id = t.card_id
WHERE t.card_id IS NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

targetes_no_usades
0

Result 9

Output

Action Output

#	Time	Action	Message
1	11:58:47	SELECT COUNT(*) AS targetes_no_usades FROM credit_cards cc LEFT JOIN transactions t ON cc.id = t...	1 row(s) returned

Creació de la taula

```sql
-- Creem la taula
CREATE TABLE estat_targetes AS
SELECT card_id,
CASE
    WHEN SUM(declined) = 3 THEN 'Inactiva'
    ELSE 'Activa'
END AS estat_targeta
FROM (
    SELECT
        card_id,
        timestamp,
        declined,
        ROW_NUMBER() OVER (
            PARTITION BY card_id
            ORDER BY timestamp DESC, id DESC
        ) AS num_transaccio
    FROM transactions
) AS transaccions_numerades
WHERE num_transaccio <= 3
GROUP BY card_id;
```

Output

Action Output

#	Time	Action	Message
1	11:58:47	SELECT COUNT(*) AS targetes_no_usades FROM credit_cards cc LEFT JOIN transactions t ON cc.id = t...	1 row(s) returned
2	11:59:35	CREATE TABLE estat_targetes AS SELECT card_id, CASE WHEN SUM(declined) = 3 THEN 'Inactiva' ...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

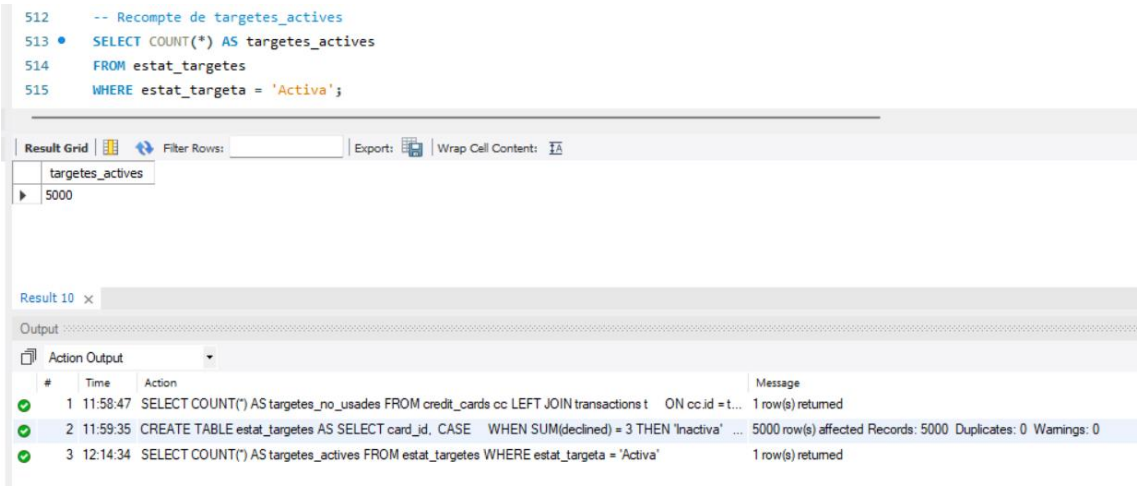
Per tal de crear la taula per determinar si les targetes són actives o no s'ha seguit aquest procés:

1) Crear una subconsulta que permeti identificar i numerar les transaccions de cada targeta. Això ho fem mitjançant PARTITION BY, que permet crear grups per fer càlculs, però no divideix el resultat en consultes independents, i ROW_NUMBER(), que permet assignar un número a cada fila dins una partició per a cada conjunt de resultats.
2) Crear una consulta externa que agrupi per targeta i amb un filtre que permeti quedar-nos només amb les 3 últimes transaccions de cada targeta
3) Utilitzar CASE per classificar les targetes com a activa o inactiva amb CASE. Com que es considera inactiva si les últimes 3 transaccions han estat declinades (declined=1) s'utilitza el SUM per determinar les inactives i la resta classificar-les com actives.

   Aclariment: Les condicions de l'enunciat són per targetes que tinguin almenys tres transaccions. En cas de tenir-ne només 1 o 2, es considerarà activa, ja que no pot complir la condició de que les últimes 3 hagin estat rebutjades.

4) Finalment crear la nova taula amb CREATE TABLE.

Recompte de targetes actives



Ens dona com a resultat que hi ha 5000 targetes actives (és a dir, totes les targetes).

Per assegurar que realment no hi ha targetes inactives, mirem si realment alguna s'ha arribat a rebutjar 3 vegades

```sql
-- Comprovació que realment no hi ha targetes inactives
SELECT card_id, SUM(declined) AS num_rebutjos
FROM (
    SELECT
        card_id,
        declined,
        ROW_NUMBER() OVER (
            PARTITION BY card_id
            ORDER BY timestamp DESC, id DESC
        ) AS num_transaccio
    FROM transactions
) AS transaccions_numerades
WHERE num_transaccio <= 3
GROUP BY card_id
ORDER BY num_rebutjos DESC
LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

card_id	num_rebutjos
CcS-9406	2
CcS-4969	1
CcS-5346	1
CcS-5006	1
CcS-5266	1
CcS-4957	1
CcS-5110	1

Result 12

Output

Action Output

#	Time	Action	Message
1	12:17:02	SELECT card_id, SUM(declined) AS num_rebutjos FROM (SELECT card_id, declined, RO...	10 row(s) returned

Confirmem que, de fet, només hi ha una targeta que s'ha rebutjat en les dues últimes transaccions, però cap a fallat a les 3 últimes.

Exercici 2

Crea una taula amb la qual puguem unir les dades de l'arxiu de products.csv amb la base de dades creada (ja que fins ara no podíem fer-ho), tenint en compte que des de transaction tens product_ids. Genera la següent consulta: Necessitem conèixer el nombre de vegades que s'ha venut cada producte.

PAS 1: Creació de la taula products i importació de les dades

Aclariment: els fitxers CSV originals són necessaris per reproduir la base de dades. Les rutes de LOAD DATA LOCAL INFILE corresponen a l'entorn de desenvolupament i s'han d'adaptar a la ubicació dels CSV. Cal tenir habilitada l'opció local_infile al client i al servidor MySQL.

Creació de la taula "products"

```
541     -- PAS 1: Creació de la taula products i importació de les dades
542 •   CREATE TABLE products (
543         id INT PRIMARY KEY,
544         product_name VARCHAR(50),
545         price VARCHAR(30),
546         colour VARCHAR(20),
547         weight DECIMAL (10, 1),
548         warehouse_id VARCHAR(20),
549         category VARCHAR(20),
550         brand VARCHAR(20),
551         cost VARCHAR (30),
552         launch_date DATE
553         );
554
```

Output

Action Output

#	Time	Action	Message
1	12:21:42	CREATE TABLE products (id INT PRIMARY KEY, product_name VARCHAR(50), price VARCHAR(30)...	0 row(s) affected

Mantenim alguns camps com a VARCHAR per facilitar la importació, especialment els imports que contenen el símbol de moneda.

S'hi introdueixen les dades de l'arxiu "products"

```
555 •   LOAD DATA LOCAL INFILE 'C:/Users/clara/Documents/itacademy/S2/N1-Ex.8__products.csv'
556     INTO TABLE products
557     FIELDS TERMINATED BY ','
558     OPTIONALLY ENCLOSED BY '"'
559     LINES TERMINATED BY '\n'
560     IGNORE 1 LINES;
561
562     -- Comprovació de que s'ha carregat bé
563 •   SELECT COUNT(*) FROM products;
```

Output

Action Output

#	Time	Action	Message
1	12:21:42	CREATE TABLE products (id INT PRIMARY KEY, product_name VARCHAR(50), price VARCHAR(30)...	0 row(s) affected
2	12:23:18	LOAD DATA LOCAL INFILE 'C:/Users/clara/Documents/itacademy/S2/N1-Ex.8__products.csv' INTO TABL...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0

Comprovació de que s'ha carregat bé

```
562     -- Comprovació de que s'ha carregat bé
563 •   SELECT COUNT(*) FROM products;
564 •   SELECT * FROM products LIMIT 10;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	product_name	price	colour	weight	warehouse_id	category	brand	cost	launch_date
1	Direwolf Stannis	$161.11	#7c7c7c	1.0	WH-4	Sports	Runway Pro	$63.84	2014-11-01
2	Tarly Stark	$9.24	#919191	2.0	WH-3	Gaming	Questbyte	$4.74	2013-02-07
3	duel tourney Lannister	$171.13	#d8d8d8	1.5	WH-2	Food	Daily Pantry	$107.07	2013-08-18
4	warden south duel	$71.89	#111111	3.0	WH-1	Books	NorthPress	$39.17	2013-12-31
5	skywalker ewok	$171.22	#dbdbdb	3.2	WH-0	Home	NordHaus	$83.69	2014-07-17

Result 13 products 14

Output

Action Output

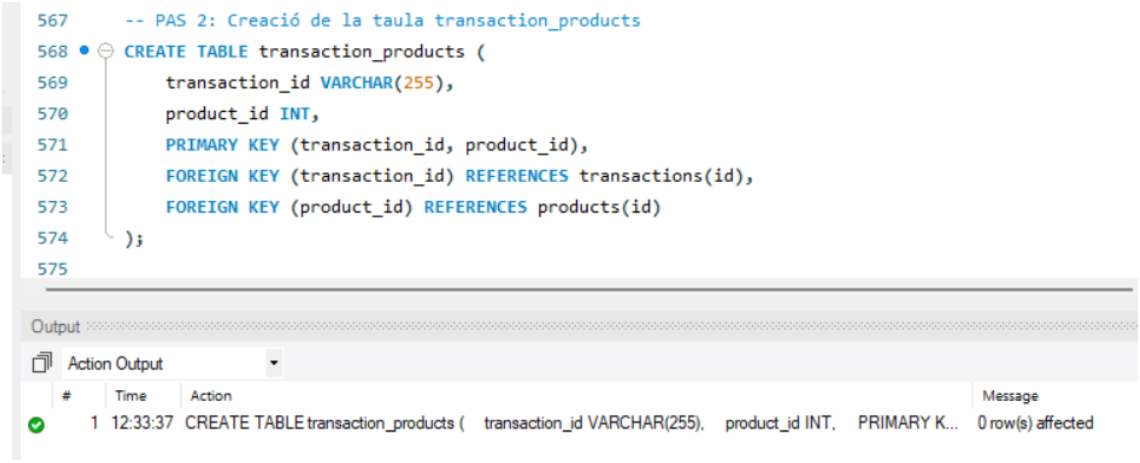
#	Time	Action	Message
1	12:21:42	CREATE TABLE products (id INT PRIMARY KEY, product_name VARCHAR(50), price VARCHAR(3...	0 row(s) affected
2	12:23:18	LOAD DATA LOCAL INFILE 'C:/Users/clara/Documents/itacademy/S2/N1-Ex.8__products.csv' INTO TAB...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0
3	12:25:19	SELECT COUNT(*) FROM products	1 row(s) returned
4	12:25:19	SELECT * FROM products LIMIT 10	10 row(s) returned

PAS 2: Creació de la taula transaction_products

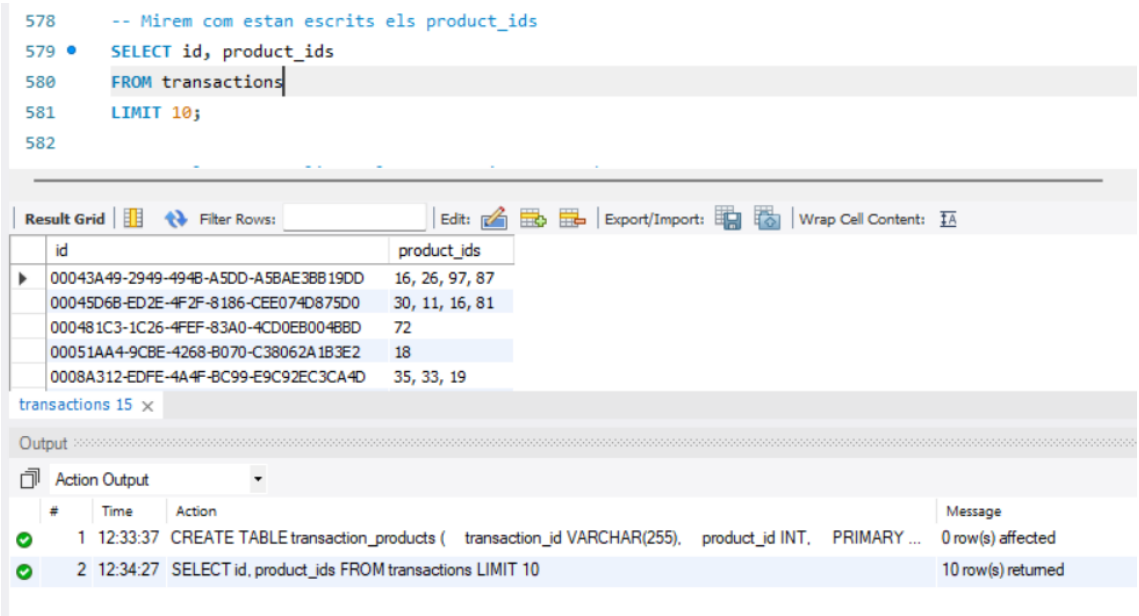
Per tal de poder fer la consulta que ens demanen necessitem crear una taula intermèdia, ja que la relació entre la taula de productes i de transaccions seria de molts a molts (N:N) i ens convé una taula on hi aparegui repetida cada transacció per cada producte inclòs en aquesta.

Per fer-ho, primer utilitzarem "JSON_TABLE" per tal de convertir la línia de text del camp "product_ids" en valors JSON i "CROSS JOIN" per a que es creï un registre de cada transacció per cada producte venut.

Creació de la taula transaction_products



Comprovació de com estan escrits els productes al camp "products_ids" per poder-los transformar



Construcció i comprovació de la consulta que utilitzarem per omplir la taula, que retorna una fila per cada binomi transacció-producte

```
583  -- Consulta per omplir taula transactions x producte
584  SELECT
585      t.id AS transaction_id,
586      p.product_id
587  FROM transactions t
588  CROSS JOIN JSON_TABLE(
589      CONCAT('[', REPLACE(t.product_ids, ' ', ''), ']'),
590      '$[*]' COLUMNS (
591          product_id INT PATH '$'
592      )
593  ) AS p
594  LIMIT 20;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

transaction_id	product_id
00043A49-2949-494B-A5DD-A5BAE3BB19DD	16
00043A49-2949-494B-A5DD-A5BAE3BB19DD	26
00043A49-2949-494B-A5DD-A5BAE3BB19DD	97
00043A49-2949-494B-A5DD-A5BAE3BB19DD	87
00045D6B-ED2E-4F2F-8186-CEE074D875D0	30

Result 16

Output

Action Output

#	Time	Action	Message
1	12:33:37	CREATE TABLE transaction_products (transaction_id VARCHAR(255), product_id INT, PRIMARY ...	0 row(s) affected
2	12:34:27	SELECT id, product_ids FROM transactions LIMIT 10	10 row(s) returned
3	12:36:19	SELECT t.id AS transaction_id, p.product_id FROM transactions t CROSS JOIN JSON_TABLE(CON...	20 row(s) returned

Comprovació de que tots els productes existeixen a la taula "products" -- Correcte

```
596  -- Comprovem que tots els productes existeixen a la taula products
597  SELECT COUNT(*) AS productes_inexistents
598  FROM transactions t
599  CROSS JOIN JSON_TABLE(
600      CONCAT('[', REPLACE(t.product_ids, ' ', ''), ']'),
601      '$[*]' COLUMNS (
602          product_id INT PATH '$'
603      )
604  ) AS p
605  LEFT JOIN products pr ON p.product_id = pr.id
606  WHERE pr.id IS NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

productes_inexistents
0

Result 17

Output

Action Output

#	Time	Action	Message
1	12:33:37	CREATE TABLE transaction_products (transaction_id VARCHAR(255), product_id INT, PRIMARY ...	0 row(s) affected
2	12:34:27	SELECT id, product_ids FROM transactions LIMIT 10	10 row(s) returned
3	12:36:19	SELECT t.id AS transaction_id, p.product_id FROM transactions t CROSS JOIN JSON_TABLE(CON...	20 row(s) returned
4	12:37:59	SELECT COUNT(*) AS productes_inexistents FROM transactions t CROSS JOIN JSON_TABLE(CONCA...	1 row(s) returned

Comprovació de que no hi ha combinacions transacció-producte duplicades – no n'hi ha

```
608     -- Comprovem que no hi ha combinacions transaction-product duplicades
609     SELECT t.id, p.product_id, COUNT(*) AS repeticions
610     FROM transactions t
611     CROSS JOIN JSON_TABLE(
612         CONCAT('[', REPLACE(t.product_ids, ' ', ''), ']'),
613         '$[*]' COLUMNS (
614             product_id INT PATH '$'
615         )
616     ) AS p
617     GROUP BY t.id, p.product_id
618     HAVING COUNT(*) > 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	product_id	repeticions

Result 18

Output

Action Output

#	Time	Action	Message
1	12:33:37	CREATE TABLE transaction_products (transaction_id VARCHAR(255), product_id INT, PRIMARY ...	0 row(s) affected
2	12:34:27	SELECT id, product_ids FROM transactions LIMIT 10	10 row(s) returned
3	12:36:19	SELECT t.id AS transaction_id, p.product_id FROM transactions t CROSS JOIN JSON_TABLE(CON...	20 row(s) returned
4	12:37:59	SELECT COUNT(*) AS productes_inexistents FROM transactions t CROSS JOIN JSON_TABLE(CONCA...	1 row(s) returned
5	12:39:14	SELECT t.id, p.product_id, COUNT(*) AS repeticions FROM transactions t CROSS JOIN JSON_TABLE(C...	0 row(s) returned

Finalment, omplim la taula "transaction_products"

```
620     -- Omplim taula transaction_products
621     INSERT INTO transaction_products (transaction_id, product_id)
622     SELECT
623         t.id,
624         p.product_id
625     FROM transactions t
626     CROSS JOIN JSON_TABLE(
627         CONCAT('[', REPLACE(t.product_ids, ' ', ''), ']'),
628         '$[*]' COLUMNS (
629             product_id INT PATH '$'
630         )
631     ) AS p;
632
```

Output

Action Output

#	Time	Action	Message
1	12:40:50	INSERT INTO transaction_products (transaction_id, product_id) SELECT t.id, p.product_id FROM transa...	253388 row(s) affected Records: 253388 Duplicates: 0 Warnings: 0

S'han creat 253.388 registres, molts més dels que teníem a la taula "transactions", així que té sentit i sembla que ha anat bé.

Comprovem que la taula s'ha omplert correctament

```
633     -- Comprovem que la taula s'ha omplert correctament
634 •  SELECT *
635     FROM transaction_products
636     ORDER BY transaction_id
637     LIMIT 25;
638
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

transaction_id	product_id
00043A49-2949-494B-A5DD-A5BAE3BB19DD	16
00043A49-2949-494B-A5DD-A5BAE3BB19DD	26
00043A49-2949-494B-A5DD-A5BAE3BB19DD	87
00043A49-2949-494B-A5DD-A5BAE3BB19DD	97
00045D6B-ED2E-4F2F-8186-CEE074D875D0	11

transaction_products 23

Output

Action Output

#	Time	Action	Message
1	12:46:14	SELECT * FROM transaction_products ORDER BY transaction_id LIMIT 25	25 row(s) returned

Consulta per saber quantes vegades s'ha venut cada producte

```
639     -- Recompte productes venuts
640 •  SELECT p.id, p.product_name, COUNT(t.id) AS num_vendes
641     FROM products p
642     LEFT JOIN transaction_products tp
643         ON p.id = tp.product_id
644     LEFT JOIN transactions t
645         ON tp.transaction_id = t.id
646         AND t.declined = 0
647     GROUP BY p.id, p.product_name
648     ORDER BY num_vendes DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	product_name	num_vendes
52	riverlands the duel	2635
29	Tully maester Tarly	2607
16	the duel warden	2583
48	rock Renly in	2580
21	duel Direwolf	2579

Result 24

Output

Action Output

#	Time	Action	Message
1	12:46:14	SELECT * FROM transaction_products ORDER BY transaction_id LIMIT 25	25 row(s) returned
2	12:47:51	SELECT p.id, p.product_name, COUNT(t.id) AS num_vendes FROM products p LEFT JOIN transaction_pro...	100 row(s) returned

Utilitzem LEFT JOIN perquè volem tenir el número de vendes de tots els productes, encara que no se n'hagi venut cap, unim primer amb la taula "pont" que hem creat de transaction_products i finalment també amb "transactions" per poder descartar les declinades. Fem el recompte de transaccions (de la taula transaccions, on hi haurà només les acceptades) i agrupem per id de producte.